CANDIDATE PROJECT REPORT

COLD CHAIN MONITORING SYSTEM

Telemetry Monitoring • Excursion Detection • Corrective Action

Prepared for YottaFlex AI Recruitment Drive

72-Hour Coding Challenge | Technical Explanation and Verification Record

Prepared by: Pavilson S

November 2026

This report explains the problem, implementation choices, assumptions, validation approach, AI assistance and known limitations.

1. Executive Summary

The Cold Chain Monitoring System is a web application intended to help operational teams monitor temperature-sensitive shipments, identify conditions that may affect product quality, and record the response. It brings shipment and container information, telemetry readings, product-specific limits, alerts and corrective-action records into one workflow.

The implementation described for this challenge uses a Python FastAPI backend, PostgreSQL database and React frontend. SQLAlchemy 2.0 is used for database access, Alembic for schema migrations, and Pydantic v2 for data validation. The project also includes a deterministic excursion rule engine and a synthetic telemetry generator for repeatable testing.

What the application is intended to do

- Keep shipment, container, product and monitoring information connected.
- Evaluate readings against the applicable product policy.
- Identify out-of-range readings, implausible physical values and missing telemetry intervals.
- Create operational alerts and support acknowledgement and corrective-action tracking.
- Provide role-specific access for administrators, quality engineers, operators and planners.
- Make core behaviour easier to verify using automated tests and controlled synthetic scenarios.

How this report is organised

The following pages explain the problem and scope, workflow, architecture, technology decisions, data model, rule engine, security, frontend, validation, setup, AI use and limitations. The report is structured around the YottaFlex AI submission themes: executable software, visible work, transparent LLM use and clear reasoning.

Accuracy statement This document is based on the project details and prompt-history summary supplied for preparation. Verify every feature, command and test claim against the current repository before submission.

2. Problem Context and Objectives

2.1 Background

A cold chain is a supply chain in which products are kept within specified environmental conditions during storage and transportation. Medicines, vaccines, food and other sensitive goods may be affected when the required conditions are not maintained. Temperature readings are useful only when they are associated with the right shipment, container, time and product policy.

2.2 Problem being addressed

When monitoring is fragmented across manual logs or separate tools, teams can find it difficult to detect an excursion promptly, determine how long it continued, identify affected goods and document the response. A missing sensor reading creates another problem: without data, the system cannot safely conclude that conditions remained normal.

2.3 Project objectives

- Centralise key cold-chain operational records.
- Apply the correct threshold policy to the relevant product or shipment.
- Identify data-quality problems as well as temperature excursions.
- Preserve continuous excursion duration across consecutive violating readings.
- Support role-based access to monitoring and response workflows.
- Provide repeatable test data and verification scripts for challenge scenarios.

2.4 Intended users

Role	Primary responsibility
OT/IT Administrator	System administration, access management and audit-related permissions
Quality Engineer	Threshold policies, excursion evaluation and corrective actions
Plant Operator	Telemetry oversight and alert acknowledgement
Production Planner	Shipment planning and container assignment

Scope boundary This is a software challenge implementation using synthetic telemetry as described. Do not present it as connected to certified physical sensors or as a validated regulatory production system unless that has been separately implemented and evidenced.

3. Requirements, Assumptions and Acceptance Criteria

The candidate guidelines emphasise executable code, clear setup and verification instructions, a public GitHub repository, prompt history, and an explanation of decisions, trade-offs, tests and limitations. The project report should therefore help a reviewer reproduce the system and understand how it was developed.

3.1 Project requirements reflected in the supplied brief

Area	Requirement represented in the project
Monitoring	Record telemetry and associate it with containers and shipments.
Policy evaluation	Apply product-appropriate temperature or environmental limits.
Excursion handling	Detect threshold violations and preserve continuous duration.
Data integrity	Reject invalid lifecycle states, mismatched policies and implausible readings.
Access	Authenticate users and restrict actions by role.
Verification	Provide tests, negative-scenario validation and repeatable generated data.
Usability	Provide a React-based operational interface and API documentation.

3.2 Assumptions to state clearly

- The generator creates synthetic readings; it is not evidence of live sensor connectivity.
- The ten-minute telemetry-gap rule is a challenge-specific rule stated in the project brief.
- The physical validation boundary of -80°C to 70°C is a project anomaly check, not a universal product safety range.
- Threshold policies must be configured for the actual product and use case.
- The system is a prototype unless deployment, operational monitoring and regulatory validation are separately completed.

3.3 Acceptance criteria for a reviewer walkthrough

- A reviewer can install dependencies and start the backend and frontend using the documented steps.
- A user can authenticate and access only permitted functions.
- A shipment can be associated with a valid container and product policy.
- Normal readings are evaluated against the configured policy.
- The mandatory negative scenarios are detected or rejected as intended.
- The test and validation commands produce outputs that can be shown to the reviewer.

Evidence rule Where a criterion has not yet been verified in the current environment, label it as pending rather than claiming it has passed.

4. End-to-End Workflow

The system workflow links operational setup, monitoring, evaluation and response. The exact status labels and transitions should be confirmed in the current code.

1. Register or configure the product and its applicable threshold policy.
2. Create a shipment with the relevant movement details.
3. Assign a suitable container to the shipment.
4. Receive or generate timestamped telemetry readings for the container.
5. Validate the reading format, timestamp, physical range and related records.
6. Evaluate the reading against the applicable product policy.
7. Detect an excursion, sensor anomaly or telemetry gap and create an alert where required.
8. Allow an authorised user to acknowledge and investigate the alert.
9. Record the corrective action, responsible person, status and relevant notes.
10. Retain linked records for later review and traceability.

4.1 Example scenario

A container is assigned to a shipment carrying a product with a configured temperature range. A sequence of readings is received. If readings move outside the configured range, the rule engine identifies the excursion and tracks its duration. If readings stop arriving for more than ten minutes, the system flags a telemetry failure rather than assuming that the shipment remained safe. An authorised user reviews the alert and records the action taken.

4.2 Why traceability matters

An alert is more useful when a reviewer can trace it back to the measurement, timestamp, container, shipment and policy that caused it. The corrective-action record then explains how the team responded. This chain of evidence supports investigation and operational accountability.

Interview explanation The core workflow is not simply a temperature chart. It connects a reading to the correct shipment and policy, evaluates it, raises an issue when needed and preserves the response.

5. Architecture and Component Responsibilities

The application follows a client-server design. The React frontend communicates with FastAPI endpoints. The backend authenticates requests, validates data, applies business rules and accesses PostgreSQL through SQLAlchemy.

Component	Technology	Responsibility
Frontend	React 19, Vite, Tailwind CSS	Pages, navigation, forms and charts
API layer	FastAPI	HTTP endpoints and request handling
Schema validation	Pydantic v2	Validation and structured API data
Business services	Python services	Excursion evaluation and operational rules
Persistence	SQLAlchemy 2.0, PostgreSQL 17	Relational records and transactions
Migrations	Alembic 1.13	Versioned database schema changes
Data generation	Pandas, NumPy	Synthetic telemetry and scenario data
Testing	Pytest	Automated behaviour checks

5.1 Typical request path

11. The user submits an action in the React interface.
12. The frontend API client sends an HTTP request to FastAPI.
13. Authentication and role checks run for protected operations.
14. Pydantic validates the request data.
15. The route calls the relevant service-layer operation.
16. The service uses SQLAlchemy to read or update PostgreSQL.
17. The API returns a structured response for the frontend to display.

5.2 Why separate layers?

Separating UI, API handling, validation, business rules and persistence keeps responsibilities clear. Critical evaluation logic can be tested independently of the browser, and the frontend does not become the authority for security or business decisions.

Architecture review point Be ready to point to the actual files for one request path. A reviewer may ask you to trace a frontend action through its API route, service, model and database.

6. Technology Choices and Trade-offs

Technology	Reason for use	Trade-off or consideration
Python 3.10+	Readable backend and data tooling	Runtime and package versions must be managed.
FastAPI	Typed API routes and OpenAPI documentation	Developers must still structure business logic carefully.
SQLAlchemy 2.0	Relational ORM and connection pooling	ORM queries and session lifecycle need understanding.
Alembic	Version-controlled schema changes	Generated migrations need review before applying.
Pydantic v2	Request and response validation	Schemas must match the intended API contract.
PostgreSQL 17	Relational integrity and transactions	Requires a running database and secure configuration.
React 19 + Vite	Component-based UI and development workflow	Frontend state and API errors need deliberate handling.
Tailwind CSS	Consistent responsive styling	Utility classes need consistent organisation.
Recharts	Telemetry visualisation	Charts need clear units, timestamps and thresholds.
Pandas + NumPy	Synthetic data generation	Generated data must not be confused with real sensor data.

6.1 Key decisions

- A relational database fits the connected nature of shipments, containers, policies and alerts.
- A deterministic rule engine is easier to explain and reproduce than an opaque decision process for mandatory safety checks.
- Synthetic data makes challenge scenarios repeatable without requiring physical devices.
- A separate frontend and backend make the API independently testable.

6.2 Trade-offs to acknowledge

The challenge prioritises a locally runnable system and explainable rules. A production deployment would require additional work around device integration, monitoring, backups, operational security, scaling, data retention and validation. Those are future engineering tasks, not capabilities to imply without evidence.

7. Database Design and Integrity

The project information describes ten core SQLAlchemy models together with user and alert models. The database name supplied for local setup is cold_chain_monitoring_db. The model list and exact field definitions should be taken from the current backend source or schema documentation.

7.1 Main entities and relationships

Entity concept	Purpose	Relationship to consider
User and role	Identity and permissions	A user performs or acknowledges actions.
Product	Item being transported	A product has an applicable policy.
Shipment	Movement record	A shipment references its product and assigned container.
Container	Transport or storage unit	A container is associated with telemetry and shipment activity.
Telemetry reading	Timestamped measurement	A reading belongs to the relevant monitored unit.
Threshold policy	Permitted operating limits	A policy must match the product and be valid.
Alert / excursion	Detected issue	An alert links to the evidence that triggered it.
Corrective action	Recorded response	An action is associated with an issue and responsible user.

7.2 Integrity controls

- Foreign keys protect relationships between related records.
- Lifecycle validation prevents a shipment being delivered before departure.
- Product-policy checks prevent a threshold for one product being used for another.
- Calibration and policy validity checks help prevent reliance on invalid configurations.
- Transactions should protect multi-step operations that must be completed together.

7.3 Migration management

Alembic records schema changes as migration revisions. A developer should inspect each migration, apply it to a local database and verify that the resulting schema matches the application models. A migration should not be treated as successful solely because a file was generated.

What to show Open the actual SQLAlchemy models and Alembic revision history during the interview. Explain one foreign key or constraint and the invalid data it prevents.

8. Backend API and Business Logic

The backend folder is organised into core configuration, models, routes, schemas, services and validators. This structure separates HTTP concerns from business rules and database representation.

8.1 API responsibilities

- Authentication endpoints handle sign-in and token-related operations.
- Dashboard endpoints return summary information for the interface.
- Entity endpoints manage core operational records.
- Operational endpoints support telemetry, alerts, excursion review and related workflows.

8.2 Validation and errors

Pydantic schemas validate incoming request data before business operations proceed. The service layer should also enforce rules that depend on database state, such as product-policy matching or shipment lifecycle transitions. The API should return appropriate status codes for invalid input, missing records and unauthorised actions. Internal stack traces and secrets should not be exposed to clients.

8.3 Example request walkthrough template

Step	What to identify in the repository
Frontend call	The component or service that sends the request.
Route	The FastAPI endpoint and HTTP method.
Schema	The Pydantic request and response models.
Access control	The authentication dependency and role check.
Service	The function containing the business operation.
Database	The SQLAlchemy model/query and transaction behaviour.
Response	The returned fields and possible error conditions.

8.4 API documentation

The local API documentation is expected at http://localhost:8000/docs. It can be used to inspect routes and schemas and to manually test requests. Before sharing a deployed instance, review whether the interactive documentation should remain publicly accessible.

Interview preparation Choose one implemented endpoint and be able to explain its complete path. Do not memorise only the endpoint name.

9. Excursion Rule Engine and Negative Scenarios

The supplied project details identify a deterministic ExcursionRuleEngine. It evaluates telemetry against the applicable limits and identifies readings or time periods that need attention.

9.1 Evaluation concepts

For an evaluation, the engine needs the measurement value, timestamp, monitored container or shipment context, and the applicable product policy. It should produce a result that can be explained using the observed value, expected range and evaluation time.

9.2 Continuous duration

When consecutive readings remain outside the permitted range, the duration should be calculated across the continuous event. The calculation should use timestamps and a defined rule for continuity; it should not incorrectly restart the duration at each violating sample.

9.3 Required negative scenarios

Scenario	Expected treatment stated in project details
Physical range violation	Values outside [-80, 70]°C are flagged as critical hardware anomalies.
Telemetry gap >10 minutes	Flag as critical telemetry failure; do not assume the gap was safe.
Delivered before departure	Reject through backend validation and lifecycle transition logic.
Wrong product threshold	Use foreign-key integrity and calibration/policy checks to prevent mismatches.
Excursion duration undercount	Preserve continuous duration across consecutive violating readings.

9.4 Important distinction

A physical plausibility boundary is not the same as a product's safe operating threshold. The system must distinguish an implausible sensor value from a valid measurement that is unsafe for a particular product.

Explainability For each detected issue, show the value, unit, timestamp, policy, reason, duration where relevant and affected shipment/container. This makes the result easier to investigate.

10. Frontend, Roles and Security

10.1 Frontend

The React application uses Vite, Tailwind CSS, Lucide icons and Recharts. The project notes describe 16 application pages. The interface is intended to present role-relevant workflows and monitoring information. The Axios service layer provides a central place for API configuration and authenticated requests.

- Use clear labels, units and timestamps in telemetry views.
- Show loading, empty and error states rather than presenting missing data as normal.
- Use status text as well as colour to communicate severity.
- Keep the layout usable at common desktop and tablet sizes.

10.2 Authentication

The project describes OAuth2-compatible JWT authentication. The backend verifies credentials and issues a signed token for authenticated access. Protected endpoints should validate the token on every request and reject expired or invalid tokens.

10.3 Password handling and role access

Passwords should be stored as one-way hashes using the configured Argon2 or bcrypt approach, never as plain text. Role-based access control should enforce permissions on the backend; hiding a button in the frontend is not sufficient protection.

Role	Expected access area
OT/IT Administrator	Administration and audit-related functions.
Quality Engineer	Threshold policies, excursion evaluation and CAPA actions.
Plant Operator	Telemetry oversight and alert acknowledgement.
Production Planner	Shipment dispatching and container assignment.

10.4 Security hygiene

- Keep database passwords, JWT secrets and API keys out of source control.
- Use .env for local configuration and provide a safe .env.example without real secrets.
- Change sample credentials before sharing the project.
- Use HTTPS and restrictive host/origin settings in a deployed environment.
- Avoid logging access tokens, passwords or sensitive records.

Public repository reminder The candidate FAQ specifically says not to publish secrets, credentials, personal or sensitive information. Check the repository history as well as the current files before making the repository public.

11. Synthetic Data, Testing and Verification

The generator creates synthetic telemetry and challenge scenarios. The supplied command uses 100 shipments, 30 containers and seed 20261022. A fixed seed supports repeatable test data, allowing developers to compare outputs after code changes.

11.1 Generator files

File	Purpose described in project details
generate_data.py	Generates synthetic records and telemetry.
inject_anomalies.py	Injects mandatory negative scenarios.
ground_truth.py	Tracks expected scenario information separately.
validate_data.py	Checks generated data integrity and scenario expectations.
test_reproducibility.py	Checks reproducibility using SHA-256 output checksums.

11.2 Verification commands

Pytest suite

```
$env:PYTHONPATH="backend"
.\venv\Scripts\pytest backend\tests -v
```

Negative-scenario integrity audit

```
.\venv\Scripts\python -m generator.validate_data
```

Reproducibility check

```
.\venv\Scripts\python -m generator.test_reproducibility
```

11.3 What the evidence should show

- The test command completes and its pass/fail summary is saved.
- The integrity audit identifies each mandatory scenario correctly.
- The reproducibility test reports whether generated outputs match.
- The application can be started and the key workflow can be demonstrated.

11.4 Honest reporting of results

The supplied project description lists the test and validation commands but does not include their actual output. Therefore, this report does not state a pass count or claim that every test passed. Add the real terminal output or a concise test-results table after running the commands on the final repository version.

Evidence matters YottaFlex asks candidates to show how they know the solution works. A command in a README is an instruction; its successful output is the evidence.

12. Installation, Execution and Reproduction

The following setup sequence is based on the Windows PowerShell instructions supplied with the project. Confirm paths and configuration values against the final repository.

12.1 Prerequisites

- Python 3.10 or later.
- PostgreSQL 17 running locally on port 5432, with the project database created.
- Node.js and npm for the React frontend.
- Git to clone the public repository.

12.2 Backend setup

```
cd "C:\Users\97150\Desktop\Cold Chain Monitoring"
Copy-Item .env.example .env
python -m venv venv
.\venv\Scripts\Activate.ps1
pip install -r backend\requirements.txt
cd backend
..\venv\Scripts\alembic upgrade head
cd ..
.\venv\Scripts\python -m generator.generate_data --shipments 100 --containers 30 --seed 20261022
$env:PYTHONPATH="backend"
.\venv\Scripts\uvicorn app.main:app --host 127.0.0.1 --port 8000 --reload
```

12.3 Frontend setup

```
cd "C:\Users\97150\Desktop\Cold Chain Monitoring\frontend"
npm install
npm run dev
```

12.4 Local URLs

- Backend API: http://localhost:8000
- Swagger UI: http://localhost:8000/docs
- Frontend: http://localhost:5173

Configuration safety Use local values in .env. Never commit the actual .env file or real credentials. Keep only placeholder variable names and safe example values in .env.example.

13. AI Assistance and Prompt History

The candidate guidelines allow LLM tools but ask candidates to explain where AI accelerated the work, where they relied heavily on it, and what they personally reviewed, changed or validated. The FAQ also states that the project must remain the candidate's individual work and that the candidate must understand the core logic, design decisions, dependencies and limitations.

13.1 AI tool disclosure

Item	Disclosure
Tool identified in supplied history	Antigravity (Google DeepMind)
Role	Supporting development assistant for technical exploration, architecture discussions, code suggestions, debugging, schema/migration guidance, synthetic data, testing ideas and frontend development.
Developer responsibility	Requirements interpretation, reviewing suggestions, integration decisions, local execution and verification of behaviour.
Verification approach	Local Windows, PostgreSQL and Node.js environment, plus the stated test and validation commands. Actual outputs should be retained as evidence.

13.2 Where AI assisted

- Discussing architecture and breaking the challenge into implementation stages.
- Suggesting approaches for models, schemas, migrations and API organisation.
- Helping explore deterministic rule-engine logic and edge cases.
- Suggesting synthetic-data generation and test scenarios.
- Supporting frontend layout and component implementation.

13.3 What must be personally understood and checked

- The actual code that was accepted, edited or rejected.
- Database relationships, constraints and migration effects.
- Authentication, password hashing and role checks.
- Excursion duration, missing-interval and physical-boundary logic.
- Test results, known failures and limitations.

13.4 Prompt history completeness

The prompt history supplied for this report is a trajectory summary: it lists the main stages from the initial challenge prompt through environment setup, backend, rule engine, data generation, frontend and verification. It is not a verbatim export of every prompt submitted to every AI tool.

Required before submission YottaFlex asks for all prompts submitted to every AI tool used. Attach the complete prompt log where available. If a tool cannot export its history, keep a reasonable record and explicitly disclose that limitation. Do not label this summary as the complete prompt history.

14. Limitations, Next Steps and Interview Walkthrough

14.1 Known limitations to state

- The supplied details describe synthetic telemetry; physical sensor integration is not established.
- The exact test pass count and validation output were not included in the supplied project information.
- The prompt history currently available is a summary, not the complete verbatim prompt export.
- Production deployment, regulatory validation, backup strategy and operational monitoring are not evidenced by the supplied details.
- The precise model names, endpoint list and all 16 page names should be confirmed from the current source code.

14.2 Practical next steps

18. Run the complete setup from a clean environment and record the steps that work.
19. Run Pytest, the negative-scenario audit and reproducibility check; save the actual output.
20. Review the public GitHub repository for secrets, credentials and accidental personal data, including commit history.
21. Add the full prompt history from every AI tool used and describe what was personally reviewed or changed.
22. Check README instructions, dependency versions, database configuration and sample data.
23. Prepare a short live walkthrough using one normal case and at least two negative scenarios.

14.3 Suggested reviewer walkthrough

Order	Demonstration
1	Show the public repository, README and meaningful commit history.
2	Explain the architecture and why the main technologies were selected.
3	Start the backend and frontend using the documented instructions.
4	Sign in and show role-appropriate application access.
5	Demonstrate a normal telemetry reading and an excursion or gap scenario.
6	Show the test output and explain what each validation proves.
7	Explain AI assistance honestly, the code you reviewed and the remaining limitations.

14.4 Closing summary

The Cold Chain Monitoring System demonstrates a full-stack approach to cold-chain visibility and exception handling. Its key engineering focus is connecting telemetry to the right operational records, applying the correct policy, detecting missing or abnormal data, and retaining a traceable response. The strongest interview discussion will come from explaining the actual implementation and showing repeatable evidence rather than making broad claims.

Final reminder The YottaFlex review values sound judgement, working software, honest AI disclosure and clear communication. Be precise about what is implemented, what has been tested, what is simulated and what remains incomplete.